

Singleton — Bilingual Study Document (English + Hindi)

Following the section order of refactoring.guru/design-patterns/singleton — written in original wording with self-authored PHP examples

Study Companion · Updated July 2026

Singleton

Note on sourcing / स्रोत पर टिप्पणी: refactoring.guru is a copyrighted commercial site (it sells a companion ebook and course). This document follows the same section order as their Singleton page — Intent → Problem → Solution → Real-World Analogy → Structure → Pseudocode → Applicability → How to Implement → Pros and Cons → Relations with Other Patterns → Code Examples — but every explanation below is written fresh in my own words rather than quoted, and all PHP code is original.

refactoring.guru एक कॉपीराइटेड (copyrighted) कमर्शियल साइट है (यह एक साथी ईबुक और कोर्स बेचती है)। यह दस्तावेज़ उनके Singleton पेज जैसा ही सेक्शन-क्रम फ़ॉलो करता है — Intent → Problem → Solution → Real-World Analogy → Structure → Pseudocode → Applicability → How to Implement → Pros and Cons → Relations with Other Patterns → Code Examples — लेकिन नीचे दी गई हर व्याख्या कोट (quote) करने के बजाय मेरे अपने शब्दों में ताज़ा लिखी गई है, और सारा PHP कोड ओरिजिनल है।

Intent

Singleton is a creational design pattern that guarantees a class has only one instance in the running process, and gives the rest of the program one well-known place to reach that instance from.

Singleton एक क्रिएशनल (creational) डिज़ाइन पैटर्न है, जो यह गारंटी देता है कि चल रही प्रक्रिया (running process) में किसी क्लास का सिर्फ़ एक ही instance मौजूद हो, और बाक़ी प्रोग्राम को उस instance तक पहुँचने के लिए एक जाना-पहचाना (well-known) स्थान देता है।

Problem

Singleton is really answering two questions at once, and that's worth naming up front because it's also the pattern's biggest structural criticism: it bundles two responsibilities into one class.

Singleton असल में एक साथ दो सवालों के जवाब देता है, और इसे शुरुआत में ही नाम देना ज़रूरी है क्योंकि यही इस पैटर्न की सबसे बड़ी संरचनात्मक (structural) आलोचना भी है: यह दो ज़िम्मेदारियों (responsibilities) को एक ही क्लास में बाँध देता है।

The first question is: how do you guarantee a class has exactly one instance? Think about why that would ever matter — usually because several parts of the program need to share access to one resource (a database connection, a single configuration set, a single log file) and having several independent instances of the class managing that resource would let them drift out of sync with each other. A normal constructor can't help here, because by definition calling a constructor always builds a brand-new object — there's no way for a plain `new` call to say "actually, here's the one that already exists."

पहला सवाल है: आप यह गारंटी कैसे दें कि किसी क्लास का सिर्फ़ एक ही instance है? ज़रा सोचिए यह कब मायने रखेगा — आमतौर पर तब, जब प्रोग्राम के कई हिस्सों को एक ही रिसोर्स (जैसे डेटाबेस कनेक्शन, एक कॉन्फ़िगरेशन सेट, या एक लॉग फ़ाइल) तक साझा (shared) पहुँच चाहिए हो, और अगर उस रिसोर्स को संभालने वाले क्लास के कई स्वतंत्र (independent) instances हों, तो वे एक-दूसरे से अलग (out of sync) हो सकते हैं। एक सामान्य कंस्ट्रक्टर यहाँ मदद नहीं कर सकता, क्योंकि परिभाषा के अनुसार कंस्ट्रक्टर कॉल करने पर हमेशा एक बिल्कुल नया ऑब्जेक्ट ही बनता है — एक सामान्य `new` कॉल यह नहीं कह सकता कि "असल में, यह वही है जो पहले से मौजूद है।"

The second question is: how do you give the rest of the program easy access to that one instance, without resorting to a plain global variable? Plain globals work, but they're dangerous — literally any piece of code, anywhere, can silently overwrite them, and there's no single place responsible for protecting that value's integrity. Singleton gives you the *convenience* of a global (reach it from anywhere) while keeping the *protection* a well-encapsulated class provides (nothing outside the class can silently swap out the instance).

दूसरा सवाल है: बिना एक सामान्य ग्लोबल वैरिएबल (global variable) का सहारा लिए, आप बाक़ी प्रोग्राम को उस एक instance तक आसान पहुँच कैसे दें? सामान्य ग्लोबल्स काम तो करते हैं, लेकिन ख़तरनाक होते हैं — कोई भी कोड, कहीं से भी, चुपचाप उन्हें बदल सकता है, और उस वैल्यू की अखंडता (integrity) की रक्षा के लिए कोई एक ज़िम्मेदार जगह नहीं होती। Singleton आपको एक ग्लोबल की सुविधा (कहीं से भी पहुँचना) देता है, साथ ही एक अच्छी तरह एनकैप्सुलेटेड (encapsulated) क्लास की सुरक्षा भी बनाए रखता है (क्लास के बाहर कोई भी चुपचाप instance को बदल नहीं सकता)।

Because the pattern has become so well-known, people sometimes call something "a singleton" loosely, even if it only satisfies one of these two goals rather than both — worth knowing so the term doesn't get used too strictly or too loosely in conversation with an interviewer.

क्योंकि यह पैटर्न इतना जाना-पहचाना हो गया है, लोग कभी-कभी किसी चीज़ को ढीले तरीक़े से "एक सिंगलटन" कह देते हैं, भले ही वह इन दोनों में से सिर्फ़ एक ही लक्ष्य पूरा करती हो, दोनों नहीं — यह जानना ज़रूरी है ताकि इंटरव्यूअर से बात करते समय यह शब्द न तो बहुत सख्ती से और न ही बहुत ढीले तरीक़े से इस्तेमाल हो।

Solution

Every Singleton implementation shares the same two ingredients.

हर Singleton इम्प्लीमेंटेशन में एक जैसी दो चीज़ें होती हैं।

First, the default constructor is made private. This stops any code outside the class from creating a new instance with the `new` keyword — the only code that's still allowed to call the constructor is the class itself.

पहला, डिफ़ॉल्ट कंस्ट्रक्टर को प्राइवेट (private) बना दिया जाता है। इससे क्लास के बाहर का कोई भी कोड `new` कीवर्ड से नया instance नहीं बना सकता — सिर्फ़ क्लास खुद ही अपने कंस्ट्रक्टर को कॉल कर सकती है।

Second, a static "get-the-instance" method is added, and it behaves like a substitute constructor. Internally, the first time it's called, it calls the now-private constructor once and stores the result in a static field belonging to the class (not to any one instance). Every call after that simply hands back the object that's already stored there, instead of building a new one.

दूसरा, एक स्टैटिक (static) "instance-लाओ" मेथड जोड़ा जाता है, जो एक विकल्प (substitute) कंस्ट्रक्टर की तरह काम करता है। अंदर से, पहली बार कॉल होने पर, यह अब-प्राइवेट कंस्ट्रक्टर को एक बार कॉल करता है और परिणाम को क्लास के एक स्टैटिक फ़ील्ड में (किसी एक instance का नहीं, बल्कि क्लास का) स्टोर कर देता है। इसके बाद की हर कॉल, नया ऑब्जेक्ट बनाने के बजाय, बस वही ऑब्जेक्ट लौटा देती है जो पहले से स्टोर है।

As long as any part of the program has access to the class, it has access to that static method — and every single call to it, from anywhere, resolves to the exact same object.

जब तक प्रोग्राम के किसी भी हिस्से को उस क्लास तक पहुँच है, उसे उस स्टैटिक मेथड तक भी पहुँच है — और उसकी हर कॉल, चाहे कहीं से भी हो, बिल्कुल उसी एक ऑब्जेक्ट पर पहुँचती है।

Real-World Analogy

A national government is a natural real-world fit for this pattern. A country has exactly one currently-sitting government at a time — regardless of which specific people happen to hold office, "the Government of [Country]" is a single, well-known label that everyone refers to and that identifies the one group currently in charge.

एक राष्ट्रीय सरकार (national government) इस पैटर्न के लिए एक स्वाभाविक असल-दुनिया का उदाहरण है। किसी देश की एक समय में बिल्कुल एक ही मौजूदा सरकार होती है — चाहे पद पर मौजूद लोग कोई भी हों, "[देश] की सरकार" एक ही, जाना-पहचाना लेबल (label) है, जिसे हर कोई इस्तेमाल करता है और जो उस समय ज़िम्मेदार समूह की पहचान कराता है।

Structure

The pattern has one participant: the **Singleton** class itself. It declares a static method (commonly named `getInstance`) that always returns the same object belonging to its own class. The constructor stays hidden from outside code — calling the static method is meant to be the only route to obtaining the object.

इस पैटर्न में एक ही भागीदार (participant) होता है: **Singleton** क्लास खुद। यह एक स्टैटिक मेथड (आमतौर पर `getInstance` नाम का) डिक्लेयर करती है, जो हमेशा अपनी ही क्लास का एक जैसा ऑब्जेक्ट लौटाती है। कंस्ट्रक्टर बाहरी कोड से छुपा रहता है — ऑब्जेक्ट पाने का इकलौता रास्ता उस स्टैटिक मेथड को कॉल करना ही होना चाहिए।

Pseudocode

Picture a class that manages the app's one shared database connection. It has no public constructor at all — the only way to reach its object is through a static access method, which builds the connection once on the very first call and simply reuses it on every call afterward.

एक ऐसी क्लास की कल्पना कीजिए जो ऐप के एक साझा (shared) डेटाबेस कनेक्शन को मैनेज करती है। इसका कोई पब्लिक कंस्ट्रक्टर बिल्कुल नहीं है — उसके ऑब्जेक्ट तक पहुँचने का इकलौता रास्ता एक स्टैटिक एक्सेस मेथड है, जो पहली ही कॉल पर कनेक्शन बनाता है और उसके बाद की हर कॉल पर बस उसी को दोबारा इस्तेमाल करता है।

Original pseudocode illustrating the same idea (not copied from the source) / यही विचार दिखाने वाला ओरिजिनल पूडोकोड (स्रोत से कॉपी नहीं):

```
class DatabaseConnection is
  private static field instance: DatabaseConnection
  private constructor DatabaseConnection() is
    // open the actual connection to the database server
    // ...
  public static method getInstance(): DatabaseConnection is
    if instance is null then
      lock a mutex
      if instance is still null then // re-check inside the lock
        instance = new DatabaseConnection()
      unlock the mutex
    return instance
  public method runQuery(sql) is
    // every query in the app flows through this one method,
    // so throttling/caching can be added in exactly one place
    // ...

class App is
  method main() is
    a = DatabaseConnection.getInstance()
    a.runQuery("SELECT 1")
    b = DatabaseConnection.getInstance()
    // `b` and `a` refer to the exact same object
```

Applicability

Reach for Singleton when a class genuinely needs to have just one instance shared by every part of the program — the classic example being one database-connection object shared across the whole app rather than each module opening its own.

Singleton का उपयोग तब करें जब किसी क्लास को वाकई सिर्फ़ एक ही instance की ज़रूरत हो, जो प्रोग्राम के हर हिस्से में साझा हो — क्लासिक उदाहरण एक ऐसा डेटाबेस-कनेक्शन ऑब्जेक्ट है, जो पूरे ऐप में साझा हो, बजाय इसके कि हर मॉड्यूल अपना खुद का कनेक्शन खोले।

It's also a fit when you want firmer control than a plain global variable gives you — unlike a global, nothing outside the Singleton class itself can silently replace the cached instance with something else.

यह तब भी फिट बैठता है जब आपको एक सामान्य ग्लोबल वैरिएबल से ज़्यादा मज़बूत नियंत्रण (control) चाहिए — एक ग्लोबल के उलट, Singleton क्लास के बाहर कोई भी चुपचाप स्टोर किए गए instance को किसी और चीज़ से बदल नहीं सकता।

Worth noting: the restriction to exactly one instance isn't set in stone by the pattern itself — if a requirement changes to "exactly N instances," the only code that needs to change is inside the static access method; the rest of the calling code doesn't need to know or care.

ध्यान देने वाली बात: सिर्फ़ एक instance की यह पाबंदी पैटर्न द्वारा पत्थर की लकीर नहीं है — अगर ज़रूरत बदलकर "बिल्कुल N instances" हो जाए, तो सिर्फ़ स्टैटिक एक्सेस मेथड के अंदर का कोड बदलना होगा; बाक़ी कॉल करने वाले कोड को इसकी जानकारी या परवाह करने की ज़रूरत नहीं।

How to Implement

1. Add a private static field to the class to hold the one instance.

क्लास में एक प्राइवेट स्टैटिक फ़िल्ड जोड़ें, जो उस एक instance को रखेगी।

1. Add a public static method whose job is to hand out that instance.

एक पब्लिक स्टैटिक मेथड जोड़ें, जिसका काम उस instance को बाहर देना है।

1. Inside that method, implement lazy initialization: build the object only the first time the method is called, store it in the static field, and on every later call just return what's already stored.

उस मेथड के अंदर, लेज़ी इनिशियलाइज़ेशन (lazy initialization) इम्प्लीमेंट करें: ऑब्जेक्ट सिर्फ़ पहली बार मेथड कॉल होने पर बनाएँ, उसे स्टैटिक फ़िल्ड में स्टोर करें, और बाद की हर कॉल पर बस वही लौटाएँ जो पहले से स्टोर है।

1. Make the constructor private, so the static method remains the only code path capable of calling it.

कंस्ट्रक्टर को प्राइवेट बनाएँ, ताकि स्टैटिक मेथड ही उसे कॉल करने वाला इकलौता कोड-पथ (code path) रहे।

1. Go through the rest of the codebase and replace any direct constructor calls with calls to the new static access method.

बाक़ी कोडबेस में जाकर, कंस्ट्रक्टर की सीधी कॉल्स को नए स्टैटिक एक्सेस मेथड की कॉल्स से बदल दें।

Pros and Cons

Pros:

- You get a hard guarantee — not just a convention — that only one instance of the class can ever exist.
- Every part of the codebase gets one shared, global access point to that instance.
- The instance is only built when it's first actually needed (lazy initialization), not unconditionally at startup.

फ़ायदे:

- आपको एक पक्की गारंटी मिलती है — सिर्फ़ एक रिवाज (convention) नहीं — कि क्लास का सिर्फ़ एक ही instance कभी मौजूद हो सकता है।
- कोडबेस के हर हिस्से को उस instance तक एक साझा, ग्लोबल एक्सेस पॉइंट मिलता है।
- instance सिर्फ़ तभी बनता है जब उसकी पहली बार वाकई ज़रूरत हो (lazy initialization), स्टार्टअप पर बिना शर्त नहीं।

Cons:

- It bundles two responsibilities into one class (controlling instance count, and providing global access) — a direct tension with the Single Responsibility Principle.
- It can quietly enable bad design — parts of the program can end up knowing more about each other than they should, simply because reaching a shared instance is so easy.
- It needs extra, careful handling in a multithreaded environment, or concurrent code can end up racing to create more than one instance.
- It tends to make the calling code harder to unit test — many mocking approaches lean on being able to substitute a different implementation, and a private constructor plus a static method (which most languages don't let you override) makes that substitution awkward.

नुक़सान:

- यह दो ज़िम्मेदारियों को एक ही क्लास में बाँध देता है (instance की संख्या नियंत्रित करना, और ग्लोबल एक्सेस देना) — यह सीधे तौर पर सिंगल रिस्पॉन्सिबिलिटी प्रिंसिपल (Single Responsibility Principle) से टकराता है।
- यह चुपचाप खराब डिज़ाइन को बढ़ावा दे सकता है — प्रोग्राम के हिस्से एक-दूसरे के बारे में ज़रूरत से ज़्यादा जान सकते हैं, बस इसलिए क्योंकि एक साझा instance तक पहुँचना बहुत आसान है।
- मल्टीथ्रेडेड (multithreaded) माहौल में इसे अतिरिक्त, सावधानीपूर्वक हैंडलिंग की ज़रूरत होती है, वरना समवर्ती (concurrent) कोड एक से ज़्यादा instance बनाने की होड़ में पड़ सकता है।
- यह कॉलिंग कोड को यूनिट-टेस्ट करना मुश्किल बना देता है — कई मॉकिंग (mocking) तरीक़े किसी दूसरे इम्प्लीमेंटेशन से बदलने की क्षमता पर निर्भर करते हैं, और एक प्राइवेट कंस्ट्रक्टर के साथ एक स्टैटिक मेथड (जिसे ज़्यादातर भाषाएँ ओवरराइड नहीं करने देतीं) इस बदलाव को मुश्किल बना देता है।

Relations with Other Patterns

- A Facade class can often naturally evolve into a Singleton, since in most systems exactly one facade object is all that's ever needed.

एक Facade क्लास अक्सर स्वाभाविक रूप से Singleton में बदल सकती है, क्योंकि ज़्यादातर सिस्टम्स में सिर्फ़ एक ही facade ऑब्जेक्ट की ज़रूरत होती है।

- Flyweight can look similar to Singleton if you push all the shared state down into just one flyweight object — but two real differences remain: Singleton allows exactly one instance while a Flyweight class is designed to support many instances (each with different intrinsic state), and a Singleton's internal state is allowed to be mutable while Flyweight objects are meant to stay immutable.

Flyweight, Singleton जैसा दिख सकता है अगर आप सारी साझा (shared) स्टेट को सिर्फ़ एक flyweight ऑब्जेक्ट में समेट दें — लेकिन दो असली अंतर बने रहते हैं: Singleton बिल्कुल एक instance की अनुमति देता है, जबकि एक Flyweight क्लास कई instances को सपोर्ट करने के लिए डिज़ाइन की जाती है (हर एक अलग intrinsic स्टेट के साथ), और Singleton की आंतरिक स्टेट म्यूटेबल (mutable) हो सकती है, जबकि Flyweight ऑब्जेक्ट्स को इम्यूटेबल (immutable) रहना चाहिए।

- Abstract Factory, Builder, and Prototype objects can all, when it makes sense, be implemented as Singletons themselves.

Abstract Factory, Builder, और Prototype ऑब्जेक्ट्स को — जब ज़रूरी हो — खुद Singleton के रूप में भी इम्प्लीमेंट किया जा सकता है।

Code Examples

The examples below are original PHP code written for this study document, illustrating the same "shared database connection" scenario from the Pseudocode section above, taken through the same progression most real teaching resources use: a naive version first, then a concurrency-safe version.

नीचे दिए उदाहरण इसी अध्ययन दस्तावेज़ के लिए लिखे गए ओरिजिनल PHP कोड हैं, जो ऊपर Pseudocode सेक्शन वाले "साझा डेटाबेस कनेक्शन" परिदृश्य को उसी क्रम में दिखाते हैं जो ज़्यादातर असली टीचिंग रिसोर्सेज़ इस्तेमाल करते हैं: पहले एक नैव (naive) वर्शन, फिर एक कंकरेंसी-सेफ़ (concurrency-safe) वर्शन।

Conceptual Example — Basic Singleton

```
<?php

class DatabaseConnection
{
    private static ?DatabaseConnection $instance = null;
    private string $connectionLabel;

    // Private constructor: nothing outside this class can call `new
    DatabaseConnection()`.
    // प्राइवेट कंस्ट्रक्टर: इस क्लास के बाहर कोई भी `new DatabaseConnection()` कॉल नहीं कर सकता।
    private function __construct()
    {
        // Simulate an expensive connection setup.
        $this->connectionLabel = "conn-" . bin2hex(random_bytes(3));
    }

    public static function getInstance(): self
    {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }

    public function runQuery(string $sql): string
    {
        return "[{$this->connectionLabel}] running: {$sql}";
    }

    public function __clone()
    {
        throw new \LogicException("Cloning this Singleton is not allowed.");
    }
}

// Client code / क्लाइंट कोड
$a = DatabaseConnection::getInstance();
$b = DatabaseConnection::getInstance();

echo $a->runQuery("SELECT * FROM users") . "\n";
echo ($a === $b ? "Same instance – as expected.\n" : "Different instances – this is a
bug!\n");
```

Expected output / अपेक्षित आउटपुट:

```
[conn-3f9a1c] running: SELECT * FROM users
Same instance – as expected.
```

Real-World-Shaped Example — Guarded, Concurrency-Aware Singleton

This version adds the two guards a naive implementation usually skips (blocking `clone` and `unserialize`), plus a comment showing how to reason about thread-safety in a language-appropriate way for PHP, rather than mechanically copying a lock-based solution from a different language.

यह वर्शन वे दो सुरक्षा उपाय (guards) जोड़ता है जिन्हें एक naive इम्प्लीमेंटेशन आमतौर पर छोड़ देता है (`clone` और `unserialize` को रोकना), साथ ही एक टिप्पणी दिखाती है कि PHP के लिए भाषा-उपयुक्त (language-appropriate) तरीके से थ्रेड-सेफ्टी के बारे में कैसे सोचा जाए, बजाय किसी दूसरी भाषा से लॉक-आधारित समाधान को यांत्रिक रूप से कॉपी करने के।

```

<?php

final class ConfigRegistry
{
    private static ?ConfigRegistry $instance = null;
    private array $values;

    private function __construct(array $values)
    {
        $this->values = $values;
    }

    // Under classic PHP-FPM, each request is an isolated process, so this
    // lazy check is already safe – there's no concurrent access within one
    // request's execution. Under a long-running Swoole/RoadRunner worker,
    // this same check-then-act sequence becomes a genuine race between
    // coroutines; the safer choice there is to call bootstrap() once at
    // worker-boot time, before any request-handling coroutine starts.
    //
    // क्लासिक PHP-FPM के तहत, हर रिक्वेस्ट एक अलग-थलग (isolated) प्रोसेस है,
    // इसलिए यह लेज़ी चेक पहले से सुरक्षित है – एक रिक्वेस्ट के भीतर कोई
    // समवर्ती (concurrent) एक्सेस नहीं होता। एक लंबे समय तक चलने वाले
    // Swoole/RoadRunner वर्कर के तहत, यही चेक-देन-एक्ट क्रम coroutines के
    // बीच एक असली रेस बन जाता है; वहाँ बेहतर विकल्प है worker-boot के समय
    // एक बार bootstrap() कॉल करना, किसी भी रिक्वेस्ट-हैंडलिंग coroutine के
    // शुरू होने से पहले।
    public static function getInstance(): self
    {
        if (self::$instance === null) {
            self::$instance = new self(['app_env' => 'production']);
        }
        return self::$instance;
    }

    public function get(string $key): mixed
    {
        return $this->values[$key] ?? null;
    }

    public function __clone()
    {
        throw new \LogicException("Cloning this Singleton is not allowed.");
    }

    public function __wakeup()
    {
        throw new \LogicException("Unserializing this Singleton is not allowed.");
    }
}

// Client code / क्लाइंट कोड
$config = ConfigRegistry::getInstance();
echo "app_env = " . $config->get('app_env') . "\n";

try {
    $copy = clone $config;
} catch (\LogicException $e) {

```

```
    echo "Blocked as expected: " . $e->getMessage() . "\n";  
}
```

Expected output / अपेक्षित आउटपुट:

```
app_env = production  
Blocked as expected: Cloning this Singleton is not allowed.
```

Technical Words Glossary / तकनीकी शब्दों की शब्दावली

English Term	Hindi Translation / हिंदी अनुवाद	Example / उदाहरण
Creational Pattern	क्रिएशनल पैटर्न	Singleton, Factory और Prototype — तीनों क्रिएशनल पैटर्न हैं।
Instance	इंस्टेंस	<code>DatabaseConnection::getInstance()</code> हमेशा एक ही instance लौटाता है।
Static Field / Method	स्टैटिक फ़ील्ड / मेथड	<code>private static ?DatabaseConnection \$instance</code> — यह क्लास से जुड़ी है, किसी एक ऑब्जेक्ट से नहीं।
Private Constructor	प्राइवेट कंस्ट्रक्टर	<code>private function __construct()</code> — क्लास के बाहर से <code>new</code> कॉल नहीं की जा सकती।
Lazy Initialization	लेज़ी इनिशियलाइज़ेशन	instance सिर्फ़ पहली <code>getInstance()</code> कॉल पर बनता है, पहले से नहीं।
Global Access Point	ग्लोबल एक्सेस पॉइंट	<code>ConfigRegistry::getInstance()</code> को कोडबेस में कहीं से भी कॉल किया जा सकता है।
Single Responsibility Principle	सिंगल रिस्पॉन्सिबिलिटी प्रिंसिपल	Singleton इस प्रिंसिपल से टकराता है, क्योंकि यह दो काम एक साथ करता है।
Thread-Safety	थ्रेड-सेफ़्टी	Swoole वर्कर में <code>getInstance()</code> को थ्रेड-सेफ़ बनाना ज़रूरी हो सकता है।
Mocking (testing)	मॉकिंग (टेस्टिंग में)	यूनिट टेस्ट में असली <code>DatabaseConnection</code> की जगह एक फ़ेक (mock) वर्शन इस्तेमाल करना।
Encapsulation	एनकैप्सुलेशन	<code>\$instance</code> फ़ील्ड प्राइवेट है — यह क्लास के बाहर से सीधे बदली नहीं जा सकती।
Facade	फ़साड	Facade क्लास अक्सर Singleton के रूप में इम्प्लीमेंट की जाती है।
Flyweight	फ़्लाइवेट	Flyweight के कई instances हो सकते हैं; Singleton का सिर्फ़ एक ही।
Mutable / Immutable	म्यूटेबल / इम्यूटेबल	Singleton की स्टेट म्यूटेबल हो सकती है; Flyweight ऑब्जेक्ट्स इम्यूटेबल रहने चाहिए।
Clone (verb/ method)	क्लोन करना	<code>clone \$config</code> को <code>__clone()</code> में एक्सेप्शन थ्रो करके ब्लॉक किया गया है।

General Words Glossary / सामान्य शब्दों की शब्दावली

English Word	Hindi Meaning / हिंदी अर्थ	Example / उदाहरण
Bundle (verb)	बाँधना, समेटना	"The hotel bundled breakfast and Wi-Fi into one price." होटल ने नाश्ता और वाई-फ़ाई को एक ही कीमत में समेट दिया।
Drift (out of sync)	अलग हो जाना, बहक जाना	"The two clocks slowly drifted out of sync over the year." साल भर में दोनों घड़ियाँ धीरे-धीरे अलग-अलग समय दिखाने लगीं।
Resort to (something)	किसी चीज़ का सहारा लेना	"When the door was stuck, she resorted to kicking it open." जब दरवाज़ा फँस गया, तो उसने उसे लात मारकर खोलने का सहारा लिया।
Well-known (adjective)	जाना-पहचाना, मशहूर	"This bakery is well-known for its bread." यह बेकरी अपनी ब्रेड के लिए जानी-पहचानी है।
Loosely (adverb)	ढीले तरीक़े से, सामान्य रूप से	"People loosely call any fast car a 'sports car.'" लोग किसी भी तेज़ गाड़ी को ढीले तरीक़े से 'स्पोर्ट्स कार' कह देते हैं।
Firmer (control)	ज़्यादा मज़बूत (नियंत्रण)	"The new manager took firmer control of the budget." नए मैनेजर ने बजट पर ज़्यादा मज़बूत नियंत्रण रखा।
Set in stone	पत्थर की लकीर, पूरी तरह तय	"These plans aren't set in stone — we can still change them." ये योजनाएँ पत्थर की लकीर नहीं हैं — हम अब भी बदल सकते हैं।
Awkward	अजीब, असहज, मुश्किल	"Carrying three bags at once felt awkward." एक साथ तीन बैग उठाना अजीब-सा लग रहा था।
Tension (figurative)	खिंचाव, टकराव	"There's a tension between wanting to save money and wanting to travel." पैसे बचाने और यात्रा करने की चाह के बीच एक खिंचाव है।
Substitute (noun/verb)	विकल्प, बदलना	"We used honey as a substitute for sugar." हमने चीनी के बदले शहद का इस्तेमाल किया।

This document follows the section order of [refactoring.guru's Singleton page](#) (a copyrighted commercial site) but is written independently — original English/Hindi explanations and original PHP code — rather than reproducing their text.

यह दस्तावेज़ [refactoring.guru](#) के *Singleton* पेज (एक कॉपीराइटेड कमर्शियल साइट) के सेक्शन-क्रम को फ़ॉलो करता है, लेकिन स्वतंत्र रूप से लिखा गया है — ओरिजिनल अंग्रेज़ी/हिंदी व्याख्याएँ और ओरिजिनल PHP कोड — बजाय इसके कि उनके टेक्स्ट को दोबारा प्रस्तुत किया जाए।